

Class

1 Khái niệm

- **Class** là bản thiết kế (blueprint) mô tả trạng thái và hành vi của đối tượng.
- Từ class bạn có thể tạo các đối tượng (instances).

2 Cấu trúc và thành phần

```
class ClassName {  
    // Thuộc tính  
    int age;  
    String name;  
  
    // Constructor  
    ClassName(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    // Phương thức  
    void display() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

Thành phần:

- Thuộc tính (fields)
- Constructor (hàm khởi tạo)
- Phương thức (methods)
- Các modifiers (public, private, protected, static...)

3 Ví dụ minh họa

```
class Car {
    String brand;
    int year;
    static int totalCars = 0;

    Car(String brandName, int yearMade) {
        brand = brandName;
        year = yearMade;
        totalCars++;
    }

    void showInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }

    static void countCars() {
        System.out.println("Total cars created: " + totalCars);
    }
}

public class Demo {
    public static void main(String[] args) {
        Car c1 = new Car("Toyota", 2020);
        Car c2 = new Car("Honda", 2021);

        c1.showInfo();
        c2.showInfo();

        Car.countCars(); // prints 2
    }
}
```

4 Tính chất OOP thể hiện qua Class

- **Đóng gói (Encapsulation):** ẩn dữ liệu, truy cập thông qua phương thức công khai
- **Kế thừa (Inheritance):** lớp con kế thừa trường và phương thức của lớp cha

- **Đa hình (Polymorphism):** ghi đè (override) các phương thức để thay đổi hành vi
- **Trừu tượng (Abstraction):** che giấu chi tiết cài đặt, chỉ lộ ra giao diện cần thiết

Ví dụ:

```
class Animal {
    void sound() {
        System.out.println("Some sound...");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark!");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow!");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        a1.sound(); // Bark!
        a2.sound(); // Meow!
    }
}
```

5 Ưu điểm — Nhược điểm

Ưu điểm:

- Có thể khởi tạo đối tượng trực tiếp
- Có thể cài đặt logic hoàn chỉnh
- Hỗ trợ đầy đủ các tính chất OOP

Nhược điểm:

- Không thể quá trừu tượng nếu có nhiều lớp con cần khác biệt lớn
- Chỉ có thể kế thừa từ một lớp cha (Java)
- Có thể dẫn đến lớp “khối” lớn nếu chứa quá nhiều trách nhiệm

Abstract Class

1 Khái niệm

- **Abstract class** là lớp không thể khởi tạo trực tiếp.
- Dùng làm khuôn mẫu cho các lớp con.
- Có thể chứa phương thức trừu tượng và phương thức có thân.

2 Cấu trúc và thành phần

```
abstract class ClassName {  
    String name;  
  
    ClassName(String name) {  
        this.name = name;  
    }  
  
    abstract void display();  
  
    void info() {  
        System.out.println("This is an abstract class.");  
    }  
}
```

```
}
```

- Phương thức **abstract** (không có thân)
- Phương thức thường (có thân)
- Constructor, biến thành viên cũng được cho phép

3 Ví dụ minh họa

```
abstract class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void makeSound();
    void eat() {
        System.out.println(name + " is eating...");
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }
    void makeSound() {
        System.out.println(name + " says: Woof woof!");
    }
}

public class DemoAbstract {
    public static void main(String[] args) {
        Dog d = new Dog("Buddy");
        d.makeSound();
        d.eat();
    }
}
```

}

4 Ưu điểm — Nhược điểm

Ưu điểm:

- Cho phép chia sẻ logic chung giữa các lớp con
- Bắt buộc các lớp con phải cài đặt các phương thức trừu tượng
- Có thể hỗ trợ code mặc định + trừu tượng

Nhược điểm:

- Không thể khởi tạo trực tiếp
- Chỉ hỗ trợ kế thừa đơn (một lớp cha)
- Khi thiết kế không tốt có thể bị ràng buộc quá chặt

5 Khi nào dùng Abstract Class

- Khi các lớp con **có nhiều điểm chung** nhưng khác ở chi tiết
- Khi muốn cung cấp **một phần cài đặt sẵn** và bắt buộc lớp con hoàn thiện phần còn lại
- Khi hệ thống chỉ cần kế thừa từ một lớp chung

Ví dụ: lớp **Shape** trừu tượng, các lớp con như **Circle**, **Rectangle** cài đặt chi tiết phương thức **calculateArea**.

Interface

1 Khái niệm

- **Interface** là hợp đồng định nghĩa các phương thức mà lớp cần thực thi.

- Không chứa trạng thái (chỉ hằng số).
- Từ Java 8 có thể chứa `default` và `static` method.

2 Cấu trúc và thành phần

```
interface InterfaceName {  
    void doSomething();  
    int VALUE = 100; // public static final  
    default void show() {  
        System.out.println("Default method");  
    }  
    static void display() {  
        System.out.println("Static method in interface");  
    }  
}
```

3 Ví dụ minh họa

```
interface Flyable {  
    void fly();  
}  
  
interface Swimmable {  
    void swim();  
}  
  
class Duck implements Flyable, Swimmable {  
    public void fly() {  
        System.out.println("Duck flies low...");  
    }  
    public void swim() {  
        System.out.println("Duck swims in the lake...");  
    }  
}  
  
public class DemoInterface {  
    public static void main(String[] args) {  
        Duck d = new Duck();  
    }  
}
```

```
d.fly();  
d.swim();  
}  
}
```

4 Ưu điểm — Nhược điểm

Ưu điểm:

- Cho phép đa kế thừa hành vi (implements nhiều interface)
- Tách biệt phần định nghĩa và cài đặt
- Hỗ trợ mở rộng linh hoạt

Nhược điểm:

- Không thể chứa trạng thái (chỉ có hằng số)
- Không thể có constructor
- Khi thêm phương thức mới có thể ảnh hưởng lớp cũ (trừ khi dùng default)

5 Khi nào dùng Interface

- Khi bạn cần định nghĩa hành vi chung cho các lớp không liên quan trực tiếp
- Khi muốn một lớp có thể đảm nhiệm nhiều vai trò khác nhau
- Khi bạn thiết kế hệ thống theo kiến trúc mở, plugin, API

Ví dụ: `PaymentMethod`, `Comparable`, `Runnable`...

I. Điểm giống nhau giữa Class, Abstract Class và Interface

Tiêu chí

Giống nhau

Cấu trúc hướng đối tượng	Cả 3 đều là thành phần cơ bản của OOP , dùng để mô tả cấu trúc hoặc hành vi của đối tượng.
Đều có thể chứa phương thức	Class, Abstract Class, và Interface đều định nghĩa phương thức (methods) để biểu diễn hành vi.
Hỗ trợ kế thừa	Cả 3 đều có thể tham gia vào quan hệ kế thừa hoặc triển khai (implements) nhằm chia sẻ hoặc ép buộc hành vi.
Không thể tách rời OOP	Cả 3 đều phục vụ 4 nguyên lý chính của OOP: đóng gói, trừu tượng, kế thừa, đa hình .
Dùng để thiết kế hệ thống	Tất cả đều giúp tổ chức code có cấu trúc, dễ mở rộng, dễ bảo trì .

II. Điểm khác nhau giữa Class – Abstract Class – Interface

Tiêu chí	Class	Abstract Class	Interface
1. Mức độ trừu tượng	Cụ thể (Concrete)	Bán trừu tượng (Partial)	Hoàn toàn trừu tượng (Pure)
2. Khả năng tạo đối tượng	✅ Có thể khởi tạo (new)	❌ Không thể khởi tạo	❌ Không thể khởi tạo
3. Mục đích chính	Mô tả đầy đủ 1 đối tượng thực tế	Làm khuôn mẫu chung cho các lớp con	Định nghĩa hành vi mà các lớp phải tuân theo
4. Thành phần có thể chứa	Biến, phương thức (có thân), constructor	Biến, phương thức có thân hoặc trừu tượng, constructor	Chỉ chứa hằng số và phương thức trừu tượng (default/static từ Java 8)
5. Kế thừa	Có thể kế thừa 1 class cha	Có thể kế thừa 1 abstract class	Có thể implements nhiều interface
6. Đa kế thừa hành vi	❌ Không hỗ trợ	❌ Không hỗ trợ	✅ Hỗ trợ (đa implements)
7. Có thể chứa dữ liệu (state)	✅ Có	✅ Có	❌ Không (chỉ hằng số final static)

8. Dùng khi	Cần tạo đối tượng cụ thể	Nhiều lớp có logic chung nhưng hành vi riêng	Nhiều lớp khác nhau cần cùng “giao tiếp hành vi”
9. Từ khóa khai báo	<code>class</code>	<code>abstract class</code>	<code>interface</code>
10. Tính mở rộng	Trung bình	Cao	Rất cao

III. So sánh bản chất thiết kế

Góc nhìn	Class	Abstract Class	Interface
Về bản chất	Là đối tượng cụ thể — có dữ liệu và hành vi rõ ràng.	Là mẫu thiết kế — có thể cài đặt sẵn một phần, yêu cầu lớp con hoàn thiện.	Là hợp đồng hành vi — chỉ mô tả “phải làm gì”, không quan tâm “làm thế nào”.
Ví dụ thực tế	<code>Dog</code> , <code>Car</code> , <code>Student</code>	<code>Animal</code> , <code>Shape</code> , <code>Vehicle</code>	<code>Flyable</code> , <code>Eatable</code> , <code>Serializable</code>
Phạm vi sử dụng	Trong các chương trình cụ thể, có thể tạo ra đối tượng thực.	Dùng trong hệ thống có nhiều lớp tương tự nhau (đa hình trong cùng nhóm).	Dùng trong hệ thống cần thống nhất hành vi giữa các lớp không liên quan .